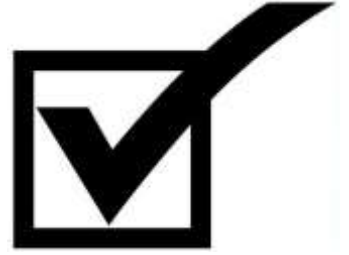




**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



C# PROGRAMMING

(330)

STATE 2023

PRODUCTION:

StateDataReader

_____ (435 points)

Test Time: 90 minutes

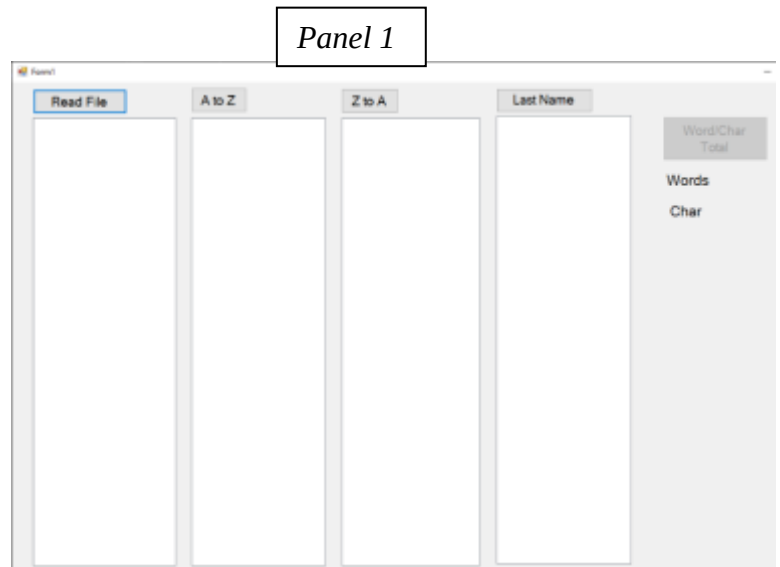
Note To Graders:

- Run the solution code a few times to get a feel for how the app should operate.
- The results from the student product need to exactly match the results from the solution.
- Check students code for evidence of hard coding the data from the .txt files; any attempt to hard code the information is automatic disqualification as it would nullify the ability to earn any points.
- The students results need to match the naming conventions exactly. Making all of the text into lowercase or uppercase for the end result that appears in the form will result in loss of points. There cannot be any changes to alpha-numeric characters in the end result.
- Conversely, if students adjust the alpha-numeric characters for manipulation and change it back to its original form for the results, this is permissible.
- Students CANNOT make any additional classes (external or internal).
- The ALL buttons cannot perform any actions until the ["Read File"] button has been pressed and reads in all of the text files.
- Students CANNOT add more objects to the Form1.
- Students will lose points for any errors that occur and cause the program to stop working. The error cannot be fixed or worked around.
- This is the naming convention for the manipulated string for the Last Name:
 - LASTNAME, FIRSTNAME, PREFIX
 - NOTE: The results will NOT be in all caps and the only items that should be observably manipulated is the order of the record.
- The total character count and total word count cannot be hard coded in the result. This will forfeit all points for that section.

Input/Output

Panel 1 shows the form that has been provided for you will appear as such when the program runs initially. All of the buttons are active; however, they will do nothing until the files have been read.

Panel 2 (below) demonstrates the four text files being read by the program when the “Read File” button is pressed. The order of the names in your list box may differ from these, however there should be 35 total names in the format shown below. And it should have the same names. Your results should match, excluding spacing.



Panel 3 (to the right) displays the output when the “A to Z” button has been pressed. All of the student records are alphabetized based upon the first letter in the records which could be the first name, or the prefix if one is present. This is intended to be a simple sorting algorithm demonstration. Your results should match, excluding spacing.



Panel 4 (on the right) is the inverse of **Panel 3** and shows the list of students being sorted in reverse alphabetical order. Again, this order is sorted based upon the first letter of the record. This is intended to be a simple sorting algorithm demonstration. Your results should match, excluding spacing.

Panel 5

Last Name

- 1) Abbott, Reilly
- 2) Arturo Altenwerth, Miss
- 3) Beahan, Irwin
- 4) Bogan, Ivah
- 5) Brooklyn Leuschke, Ms.
- 6) Carroll, Heloise
- 7) Cartwright, Payton
- 8) Christiansen, Sigurd
- 9) Colton Bergnaum, Dr.
- 10) Conn, Angel
- 11) Daniel, Jazmin
- 12) Davis, Santina
- 13) Flatley, Rowena
- 14) Gavin Tromp, Mr.
- 15) Greenholt, Filomena
- 16) Hayes, Elna
- 17) Hermiston, Alvis
- 18) Howell, Theresa
- 19) Lakin, Dallas
- 20) Lehner, Domingo
- 21) Marks, Aiyana
- 22) Marvin, Minnie
- 23) Mayer, Freeman
- 24) Mosciski, Cassandre
- 25) Nitzsche, Louisa
- 26) Reilly, Cassandre
- 27) Rossie Schultz, Dr.
- 28) Ruecker, Vern
- 29) Schaden, Shanie
- 30) Schamberger, Deron
- 31) Strosin, Yolanda
- 32) Turner, William
- 33) Veum, Yvette
- 34) Walker, Merl
- 35) Wintheiser, Emanuel

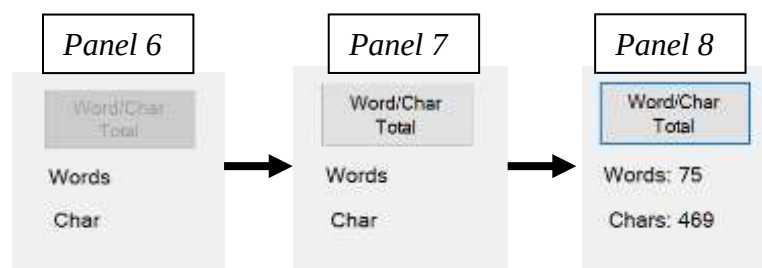
Panel 5 (on the left) will be more intensive than the other sorting tasks because you will need to reorder the student records in the following format. There are multiple ways to accomplish this goal. Your results should match, excluding spacing.

Panel 4

Z to A

- 35) Yvette Veum
- 34) Yolanda Strosin
- 33) William Turner
- 32) Vern Ruecker
- 31) Theresa Howell
- 30) Sigurd Christiansen
- 29) Shanie Schaden
- 28) Santina Davis
- 27) Rowena Flatley
- 26) Reilly Abbott
- 25) Payton Cartwright
- 24) Ms. Brooklyn Leuschke
- 23) Mr. Gavin Tromp
- 22) Miss Arturo Altenwerth
- 21) Minnie Marvin
- 20) Merl Walker
- 19) Louisa Nitzsche
- 18) Jazmin Daniel
- 17) Ivah Bogan
- 16) Irwin Beahan
- 15) Heloise Carroll
- 14) Freeman Mayer
- 13) Filomena Greenholt
- 12) Emanuel Wintheiser
- 11) Elna Hayes
- 10) Dr. Rossie Schultz
- 9) Dr. Colton Bergnaum
- 8) Domingo Lehner
- 7) Deron Schamberger
- 6) Dallas Lakin
- 5) Cassandre Reilly
- 4) Cassandre Mosciski
- 3) Angel Conn
- 2) Alvis Hermiston
- 1) Aiyana Marks

Panels 6, 7, & 8 (see below): the “Word/Char Total” button will need to be inactive (Panel 6) until the “Last Name” button is clicked and then it will become active (Panel 7). The button will need to be programmed to find the total number of words in the files and the total number of characters that exclude spaces and ‘.’ (periods), numbers, and ‘)’...only letters are counted. The totals are provided to help you determine if it was correctly analyzed.



Requirements

1. You must use the provided project called “StateDataReader”. This project will contain a “Form1.cs[design]” that has been already created in the organization and naming conventions.
2. The file “Form1.cs” has already been provided. This file contains all needed packages and the method headers have already been provided. You will program within this file.
3. No additional buttons or methods can be added to the form design.
4. Your contestant number must appear as a comment at the top of the main source code file.
5. User input (pressing the buttons) is needed for this program as the program will gain functionality through the importing and reading of four different text files.
6. You will be required to complete the given button listener methods that are already provided; you can create additional internal classes if desired, however the code can be completed without them.
7. The button that will be the catalyst to allow functionality to the other buttons is the “Read File” button. The rationale is that the files must be present in some type of data structure in order for the other buttons to be able to sort the values.
8. Records will be each individual line read by the program from the text files. Fields would be the categorization of the information in each record. For example, each record will contain the fields of prefix, first name and last name: Dr. Jose Patel.
9. The list boxes for the “A to Z” and “Z to A” buttons must sort based upon the first letter of the field in the record. In most cases this will be the first name; however some records will have a prefix (Dr. Mrs. Mr.), in these instances use the first letter of the prefix since this is the leading character in the record. These sorting functions are intended to be easy.
10. The final sorting button (“Last Name”) will require the record to be manipulated based upon one of the following naming conventions:
 - a. LASTNAME, FIRSTNAME, PREFIX
 - b. LASTNAME, FIRSTNAME
11. The “Word/Char Total” button will generate data based upon the imported information only, not the concatenated numbers. There should be 75 words (this includes the prefixes) and 469 characters that do NOT include spaces or ‘.’ (periods).
12. Hard coding the data (i.e. directly typing the data) will eliminate any chance of earning points for that particular section and associated sections. All data must be generated from the file reading and from the manipulation/analysis of the strings.
13. The program will display the EXACT outputs as shown in the examples above. NOTE: there must be spacing between each field in the record and follow the naming convention. Extra spacing of 1-2 extra values will be ignored.
14. If a scorable item on the rubric says “Comments are required” you will only be able to earn points if that item has a comment. The comment does not need to be a complete sentence, and only needs bring the graders attention to the main idea of what that method or segment of code performs. You do not need to list a comment for line by line explanation.
15. WARNING: all buttons have been named in advance and should not be change. They will be prefixed with “btn”+Name. For example, **btnReadFile**. In addition, the buttons

have an event associated with them, and those methods have already been created in advance. The naming convention for these button events which you will program are named. For example the method for the button **btnMax** has the method **btnReadFile_Click(object sender, EventArgs e) {...}** The **_Click(object sender, EventArgs e)** has been added for you. You will program these method based upon the requirements in the rubric (see next page). All buttons in this program will follow these guidelines, and you will program ALL of the buttons. A list of all buttons and methods is provided below.

- a. Button Name: **btnReadFile** Method Name: **btnReadFile_Click** Text: "Read File"
 - i. Prints to list box: **lstNames**
- b. Button Name: **btnAtoZ** Method Name: **btnAtoZ_Click** Text: "A to Z"
 - i. Prints to list box: **lstAtoZ**
- c. Button Name: **btnZtoA** Method Name: **btnZtoA_Click** Text: "Z to A"
 - i. Prints to list box: **lstZtoA**
- d. Button Name: **btnLastAtoZ** Method Name: **btnLastAtoZ_Click** Text: "Last Name"
 - i. Prints to list box: **lstLastNames**
- e. Button Name: **btnWordTotal** Method Name: **btnWordTotal_Click** Text: "Word/Char Total"
 - i. Word total prints to label: **lblWordTotal**
 - ii. Character total prints to label: **lblCharTotal**

Solution and Project		
The project is present on the flash drive		10 points
Program Execution		
The program runs from the USB flash drive: the form loads with no initial errors		15 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The program displays all buttons as active.		10 points
Before [“Read File”] button is clicked: Buttons [“A to Z”], [“Z to A”], and [“Last Name”] do NOT perform any actions when clicked. All or nothing.		10 points
The [“Read File”] button is clicked and the corresponding list box displays 35 records in random order and must match the format where the record is preceded with number and “)” for example: 1) Jose Montana.		20 points
The [“Read File”] button is clicked and the corresponding list box displays 35 records in random order and must match the format where the record is preceded with number and “)” for example: 1) Jose Montana.		20 points
The [“A to Z”] button is clicked and the corresponding list box displays 35 records in ascending order based upon the first character of the record and with the increasing numbering format 1 to 35.		20 points
The [“Z to A”] button is clicked and the corresponding list box displays 35 records in descending order based upon the first character of the record and with the decreasing numbering format 35 to 1.		20 points
The [“Last Name”] button is clicked and the corresponding list box displays 35 records in ascending order (1x) based upon the last name in the record; record has been reformatted to new naming convention (1x), commas are present (1x); count increases 1 to 35 (1x) < -10pts. per error >		40 points
The [“Word/Char Total”] button is not active until the [“Last Name”] button is pressed.		10 points
The [“Word/Char Total”] button is clicked and displays the total word count correctly (75 is the total for the provided files).		20 points
The [“Word/Char Total”] button is clicked and displays the total word character count correctly (469 is the total for the provided files).		20 points
Subtotal		/215 Points

Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present		10 points
Form1.cs class: contains uses a single list to store the imported records from the text file. Comment is required; mark this comment as “SC1” at the beginning.		10 points
Form1.cs class: contains uses a separate single list to store the reorganized records into the new format (LASTNAME, FIRSTNAME, PREFIX). The code must have worked to get credit. Comment is required; mark this comment as “SC2” at the beginning.		10 points
Form1.cs class: <i>btnReadFile_Click</i> method uses static class Path to locate the files. The code must have worked to get credit. Comment is required; mark this comment as “SC3” at the beginning.		30 points
Form1.cs class: <i>btnReadFile_Click</i> method stores all of the file pathways into an array and requires no loops to get all file locations. The code must have worked to get credit. Comment is required; mark this comment as “SC4” at the beginning.		30 points
Form1.cs class: <i>btnReadFile_Click</i> method uses static class Path to locate the files. The code must have worked to get credit. Comment is required; mark this comment as “SC4” at the beginning.		10 points
Form1.cs class: <i>btnAtoZ_Click</i> method is commented explaining how the list has been sorted. The code must have worked to get credit. Mark this comment as “SC5” at the beginning.		10 points
Form1.cs class: <i>btnZtoA_Click</i> method is commented explaining how the list has been sorted in reverse order. The code must have worked to get credit. Mark this comment as “SC6” at the beginning.		10 points
Form1.cs class: <i>btnLastAtoZ_Click</i> method is commented explaining how the records have been broken into sections. The code must have worked to get credit. Mark this comment as “SC7” at the beginning.		30 points
Form1.cs class: <i>btnLastAtoZ_Click</i> method is commented explaining how the records have been reorganized into the TWO different types of naming conventions. The code must have worked to get credit. Mark this comment as “SC8” at the beginning.		30 points
Form1.cs class: <i>btnWordTotal_Click</i> method is commented explaining the total words were counted. The code must have worked to get credit. Mark this comment as “SC9” at the beginning.		20 points
Form1.cs class: <i>btnWordTotal_Click</i> method is commented explaining the total characters were counted. The code must have worked to get credit. Mark this comment as “SC10” at the beginning.		20 points
Subtotal		/220 Points
Total Points		/435 Points

```

...PA2022\State\StateDataReader\StateDataReader\Form1.cs 1
1 using System;
2 using System.IO;
3 using System.Collections.Generic;
4 using System.ComponentModel;
5 using System.Data;
6 using System.Drawing;
7 using System.Linq;
8 using System.Text;
9 using System.Threading.Tasks;
10 using System.Windows.Forms;
11 using System.Diagnostics; //Needed for process
12
13 namespace StateDataReader
14 {
15     public partial class Form1 : Form
16     {
17         List<string> data = new List<string>();
18         List<string> dataLastName = new List<string>();
19         //List<Students> students = new List<Students>(); //This was not
           used in the solution, but it could have
20
21         public Form1()
22         {
23             InitializeComponent();
24         }
25
26         private void Form1_Load(object sender, EventArgs e)
27         {
28
29         }
30
31         // This button command will find all of the text files and read
           them into the program.
32         private void btnReadFile_Click(object sender, EventArgs e)
33         {
34             //THE TEXT FILES HAVE BEEN PLACED IN THE BIN/DEBUG FOLDER WITH
           THE EXE FILES
35             string folder = Path.GetDirectoryName
           (Process.GetCurrentProcess().MainModule.FileName); //GETS
           LOCATION OF THE EXE FILE
36
37             //NOTE: students may try to manually link every text file.
38             string filter = "*.txt"; //This is used in next line to find
           all text files
39             string[] files = Directory.GetFiles(folder, filter); //this is
           the absolute path
40             //for the file for StreamReader which normally would
           have the path in "\\path\\etc."
41

```

```
...PA2022\State\StateDataReader\StateDataReader\Form1.cs 2
42
43
44
45     string line;
46     int count = 1;
47     for(int i = 0; i < files.Length; i++)
48     {
49
50         StreamReader sr = new StreamReader(files[i]);
51         line = sr.ReadLine();
52         while (line != null)
53         {
54             data.Add(line);
55             lstNames.Items.Add(count + ") " + line);
56             line = sr.ReadLine();
57             count++;
58         }
59     }
60
61
62 }
63
64 //This command will sort the list A to Z with built in .Sort()  ➤
65     method.
66 //This sorts based upon first character of the record
67 private void btnAtoZ_Click(object sender, EventArgs e)
68 {
69     lstAtoZ.Items.Clear(); //list box should be cleared ahead but  ➤
70     not required
71     int count = 1;
72     data.Sort();
73     foreach (string s in data)
74     {
75         lstAtoZ.Items.Add(count+ ") "+s);
76         count++;
77     }
78 }
79
80 //This command will sort the list Z to A with built in Reverse()  ➤
81     method.
82 //This sorts based upon first character of the record
83 private void btnZtoA_Click(object sender, EventArgs e)
84 {
85     lstZtoA.Items.Clear();
86     int count = data.Count;
87     //data.Sort(); //Ignore.
88     data.Reverse();
89     foreach (string s in data)
90     {
```

```
...PA2022\State\StateDataReader\StateDataReader\Form1.cs 3
88         lstZtoA.Items.Add(count + ") " + s);
89         count--;
90     }
91 }
92
93 //This command will break the record into the conventions, and reorganize it.
94 private void btnLastAtoZ_Click(object sender, EventArgs e)
95 {
96     string temp1 = null;
97     string temp2 = null;
98     string temp3 = null;
99     int found = 0;
100    int count = 1;
101    foreach (string s in data)
102    {
103
104        string tempFullName = s.TrimStart();
105        temp1 = null;
106        temp2 = null;
107        temp3 = null;
108        found = tempFullName.IndexOf(" ");
109
110
111        if (temp1 == null) {
112            temp1 = tempFullName.Substring(0, found);
113            temp2 = tempFullName.Remove(0, found);
114
115
116        }
117        temp2.TrimStart();
118        found = temp2.IndexOf(" "); //Checks for an extra space which indicates a prefix was used
119        if (found > 0)
120        {
121            string temp = temp2;
122            temp2 = temp.Substring(0, found+1);
123            temp3 = temp.Remove(0, found);
124
125        }
126
127        //Prints out convention names with NO prefix
128        if (temp3 == null) {
129            dataLastName.Add(temp2 + ", "+temp1);
130        }
131        else //Prints out convention names with prefix
132        {
133            dataLastName.Add(temp3 + ", " + temp2 + " " + temp1);
134        }
135    }
136 }
```

```
...PA2022\State\StateDataReader\StateDataReader\Form1.cs 4
135
136
137     }
138
139     dataLastName.Sort();
140     foreach (string s in dataLastName)
141     {
142         lstLastName.Items.Add(count+ " " +s);
143         count++;
144     }
145     btnWordTotal.Enabled = true;
146 }
147
148 //Counts all of the words and characters in the list.
149 private void btnWordTotal_Click(object sender, EventArgs e)
150 {
151
152     int countWords = 0;
153     int countChars = 0;
154     foreach (string s in data)
155     {
156         countWords += s.Trim().Split(' ').Length;
157         countChars += s.Trim().Replace(" ", "").Replace
            (".", "").Length; //Does not count the '.' and spaces
158     }
159     lblWordTotal.Text = "Words: " + countWords.ToString();
160     lblCharTotal.Text = "Chars: " + countChars.ToString();
161
162     }
163 }
164 }
165
```